Neural Networks: From Piecewise Linear Fences to Deep Networks

Machine Learning 2 — Week 9

Index

- 1. Why Do We Need Neural Networks — SVMs were great until the boundary started looking like a plate of spaghetti.
- 2. Parameterizing a Piecewise Linear Function — building a fence out of straight planks and a bit of algebra.
- 3. The ReLU Activation Function — the simplest "if" statement in all of machine learning.
- 4. Learnable Parameters and the Training Objective — a list of knobs, and a loss that tells you how badly you turned them.
- 5. Separating the Input from the Model — drawing the algebra as a circuit, and meeting the first network.
- 6. A Worked Example with Three ReLU Units — real numbers, zigzag output, no hand-waving.
- 7. The Biological Neuron Inspiration — why it is called a neural network, and why the brain is not offended.
- 8. The Shallow Neural Network — one hidden layer, as many neurons as you can afford.
- 9. More Than One Input Feature — every neuron gets to see everything.
- 10. Layers and Terminology — input, hidden, output: the vocabulary of the field.
- 11. Going Deep by Stacking — the "obvious" way to go deep, and why nobody does it.
- 12. The Conventional Deep Network — fully connected, 59 arrows, 59 parameters.
- 13. Matrix Notation — turning a forest of equations into four tidy lines.
- 14. A Numeric Walk-Through of One Layer — matrix multiplication with every step written out.
- 15. Counting Trainable Parameters — bean counting, with a trick to check yourself.
- 16. Quick Review Sheet — the one page you read the night before.
- 17. Practice Questions — five problems that will make you think, not recall.

1. Why Do We Need Neural Networks

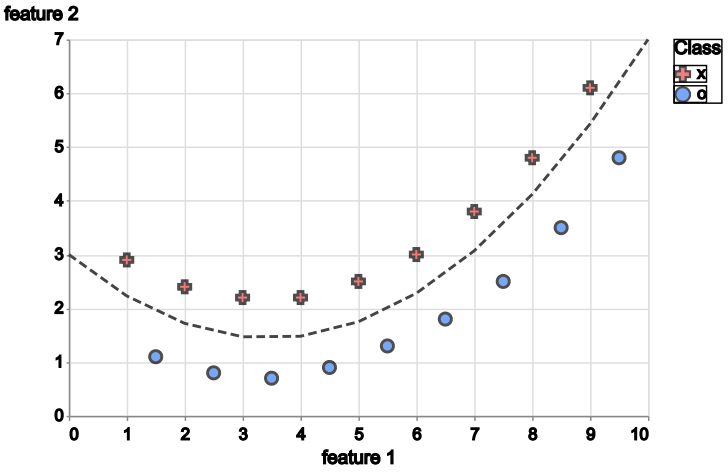
Imagine you are a surveyor who has to draw a fence between two kinds of farmland. Soil type **x** lies on one side, soil type **o** on the other. The fence cannot be a straight line, because the two regions wind around each other.

We already know one tool for this: the **kernel SVM**, which can carve out curved boundaries. But two things make it uncomfortable in general:

- The boundary can be very complex.
- The data set can have a huge number of classes (millions, say). We still want a way to train a classifier that **learns** the classification boundary from data.

Two kinds of boundary are used as running examples.

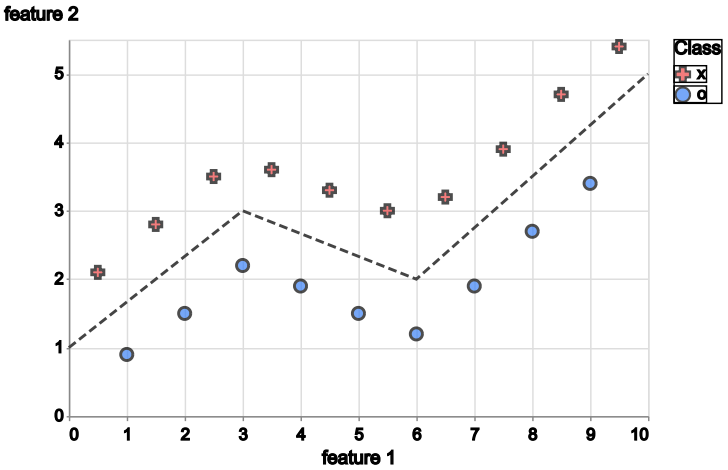
A general, continuous non-linear boundary



A piecewise linear boundary

Here the boundary is not smooth, but it is made of straight segments. Each segment separates the data in its own local region, so the whole boundary is a combination of **three linear boundaries**.

We start with the **piecewise linear** case. The question is: **how do we build such a boundary, and how do we learn it from data?**



> **Note for Review:** Both plots above are schematic. They only illustrate the *shape* of the boundaries on the lecture board; the points are not real data.

2. Parameterizing a Piecewise Linear Function

Look at the piecewise linear boundary again. In one region the boundary is a line, in the next region it is another line, and in the third region a third line. So the boundary is a **combination of 3 linear boundaries**.

We cannot train a separate classifier on each piece, because we do not know in advance where the pieces are. Instead we **parameterize one function** f so that it is automatically piecewise linear.

Let the label be predicted from the input as $y = f[x, \boldsymbol{\phi}]$, where $\boldsymbol{\phi}$ holds all the parameters of the model. For simplicity, the input x has a single feature for now (many features come later).

We create three linear functions of x and take a linear combination of them, passing each one through an activation function $a(\cdot)$:

$$y = \phi_0 + \phi_1 a(\theta_{10} + x\theta_{11}) + \phi_2 a(\theta_{20} + x\theta_{21}) + \phi_3 a(\theta_{30} + x\theta_{31})$$

Each inner expression is a linear function of x:

- $\theta_{10} + x\theta_{11}$ is the first linear classifier,
- $\theta_{20} + x\theta_{21}$ is the second,
- $\theta_{30} + x\theta_{31}$ is the third.

If the boundary has many linear pieces, we just keep adding such terms, one per piece.

> **Note for Review:** The board writes the last coefficient with the same name as the second one, and lists only ϕ_0, ϕ_1, ϕ_2 in the parameter set. With three hidden terms there must be **four** output coefficients, $\phi_0, \phi_1, \phi_2, \phi_3$. These notes use the corrected version. The same fix gives 10 parameters for this model in total.

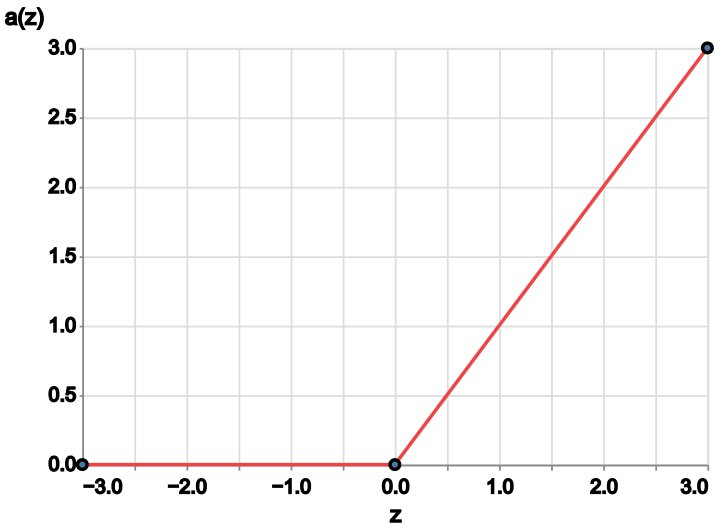
3. The ReLU Activation Function

For piecewise linear boundaries, the activation function we use is the **ReLU** (Rectified Linear Unit), the most widely used activation function:

$$a(z) = \text{ReLU}(z) = \begin{cases} z & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Before 0 the value is always 0. After 0 it rises linearly with slope

Why ReLU keeps boundaries piecewise linear. ReLU is itself piecewise linear (two pieces, joined at $z = 0$). Passing a linear function through it therefore does **not** turn a linear classifier into a smooth non-linear one. The result is still piecewise linear, with a single bend (a "kink") where the input to ReLU crosses zero.



ReLU as a hinge. For one hidden unit, $a(\theta_{d0} + x\theta_{d1})$ is flat ($= 0$) until

$$x = -\frac{\theta_{d0}}{\theta_{d1}}$$

and after that it grows with slope θ_{d1} (for $\theta_{d1} > 0$). Each hidden unit adds one bend to the function, and the output coefficient ϕ_d scales how steep that bend is, and flips it if $\phi_d < 0$.

> **Note for Review:** *Pro-Tip:* If ReLU were replaced by the identity $a(z) = z$, the whole expression would collapse to one straight line, because a sum of linear functions of x is linear. The non-linearity of a is what lets the units "bend" the output.

4. Learnable Parameters and the Training Objective

The model is

$$y = f[x, \boldsymbol{\phi}]$$

where the parameter vector collects every learnable number. For three hidden units:

$$\boldsymbol{\phi} = \{\phi_0, \phi_1, \phi_2, \phi_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}\}$$

Each different choice of these numbers gives a **different** piecewise linear function. So the boundary's shape changes as the parameters change, and depending on the data set we must **learn** the parameters to get the best boundary.

Training. Suppose we are solving a (piecewise linear) regression or classification problem with n data points. For the i-th point, the prediction is

$$\hat{y}_i = f[x_i, \boldsymbol{\phi}]$$

and the ground-truth label is y_i. The loss over the data set is

$$\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

and training means minimizing it over all the parameters, using any of the optimization techniques we have studied:

$$\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2 \;\Rightarrow\; \min_{\boldsymbol{\phi}} \frac{1}{n}\sum_{i=1}^{n}(f[x_i, \boldsymbol{\phi}] - y_i)^2$$

The minimizer gives the optimal parameters for the given data set.

> **Note for Review:** The symbol $\boldsymbol{\phi}$ is used in two ways in the notes: as the **whole parameter set** (bold, $\boldsymbol{\phi}$) and as the **output-layer weights** $\phi_0, \phi_1, \ldots$ (plain, with an index). Context makes it clear, but watch out when reading equations.

5. Separating the Input from the Model

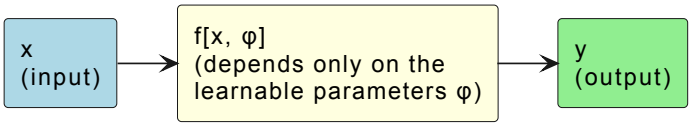
Managing a long expression with many terms is tedious. Keep two hidden terms to simplify, and give the inner pieces names:

$$\begin{aligned} y &= \phi_0 + \phi_1 a(\theta_{10} + x\theta_{11}) + \phi_2 a(\theta_{20} + x\theta_{21}) \\ &= \phi_0 + \phi_1 h_1 + \phi_2 h_2 \end{aligned}$$

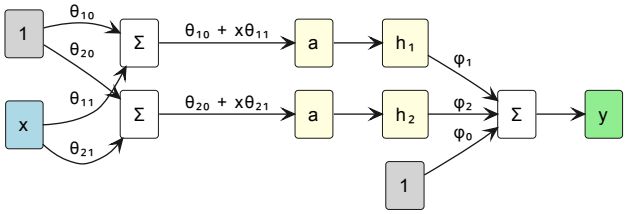
with

$$h_1 = a(\theta_{10} + x\theta_{11}), \qquad h_2 = a(\theta_{20} + x\theta_{21})$$

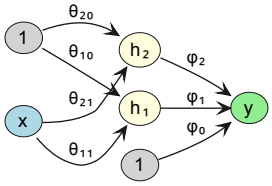
Now x is the input and y is the output, and the box between them depends only on the learnable parameters $\boldsymbol{\phi}$:



So what is inside the box? Look at the first term. The constant input 1 is multiplied by θ_{10}, the input x is multiplied by θ_{11}, and the two are **added**. The sum $\theta_{10} + x\theta_{11}$ is passed through the activation a, which gives h_1. The same happens for h_2 with θ_{20}, θ_{21}. Finally h_1 is multiplied by ϕ_1, h_2 by ϕ_2, a constant 1 is multiplied by ϕ_0, and everything is added to give y.



Since the activation is applied at every unit, we stop drawing a as a separate box and fold it into the unit. That gives the compressed picture, which is our **first neural network**:



> **Note for Review:** This network has **one** layer of intermediate units (the h's), which is why it is called a **shallow** neural network.

---

6. A Worked Example with Three ReLU Units

Take three hidden units and choose these parameters:

Parameter	θ_{10}	θ_{11}	θ_{20}	θ_{21}	θ_{30}	θ_{31}	ϕ_0	ϕ_1	ϕ_2	ϕ_3
Value	0	1	−1	1	−2	1	0	1	−2	2

Then

$$h_1 = \mathrm{ReLU}(x), \qquad h_2 = \mathrm{ReLU}(x-1), \qquad h_3 = \mathrm{ReLU}(x-2)$$

$$y = 0 + 1 \cdot h_1 - 2\,h_2 + 2\,h_3$$

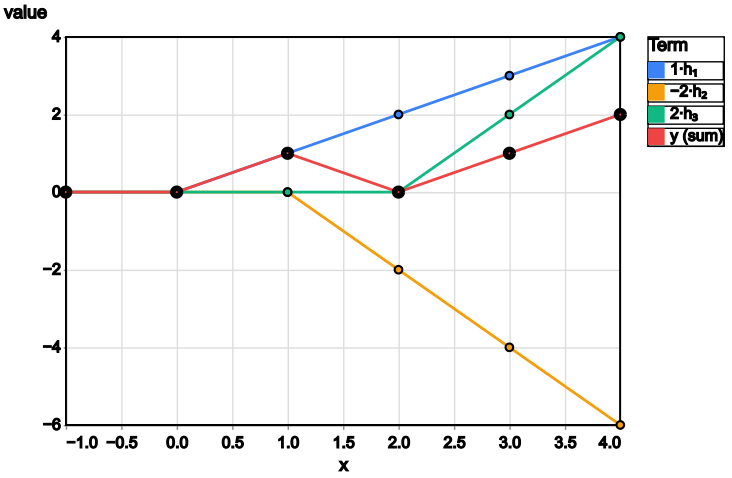
The kinks are at $x = 0, x = 1, x = 2$ (from $-\theta_{d0}/\theta_{d1}$). Evaluating step by step:

x	$z_1 = x$	$z_2 = x-1$	$z_3 = x-2$	h_1	h_2	h_3	$y = h_1 - 2h_2 + 2h_3$
−1	−1	−2	−3	0	0	0	0
0	0	−1	−2	0	0	0	0
1	1	0	−1	1	0	0	1
2	2	1	0	2	1	0	$2-2+0=0$
3	3	2	1	3	2	1	$3-4+2=1$
4	4	3	2	4	3	2	$4-6+4=2$

Slope of each piece of y:

Region	Active units	Slope of y
$x < 0$	none	0
$0 < x < 1$	h_1	1
$1 < x < 2$	h_1, h_2	$1-2=-1$
$x > 2$	h_1, h_2, h_3	$1-2+2=1$

The thick red line is the final piecewise linear function. Each ReLU contributes one bend, and the output weights decide whether the bend goes up or down.



7. The Biological Neuron Inspiration

Why are these things called **neural** networks?

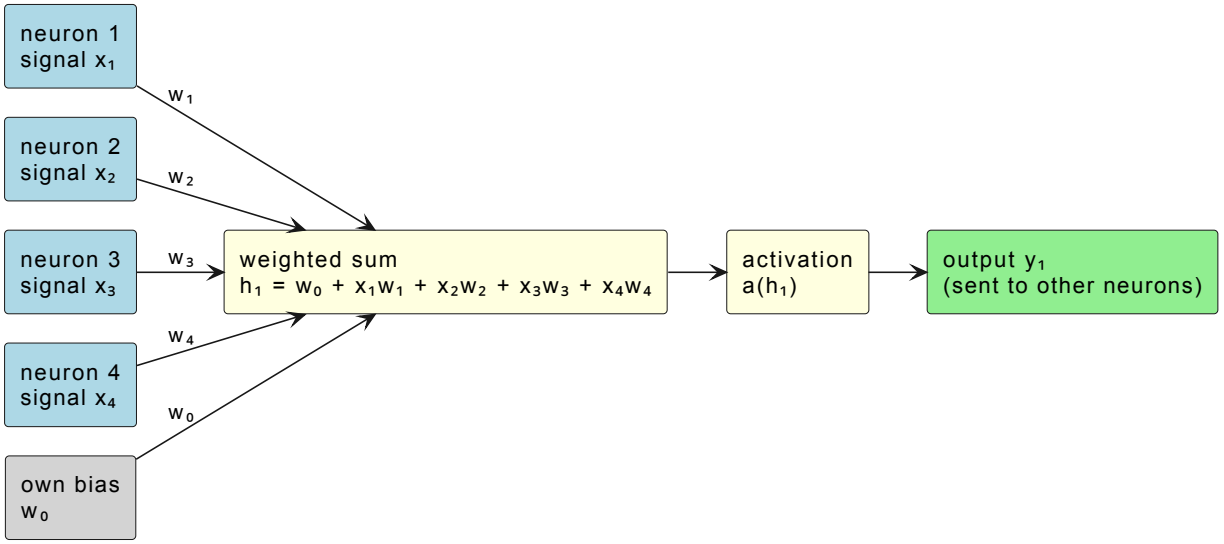
There are billions of neurons in the brain, interconnected with each other, and together they process information from the sensory organs. A single neuron:

1. **Receives** information from other neurons through action potentials.
2. At the **synapses**, neurotransmitters allow the action potential, which carries the information, to pass from one neuron to the next. The *density of neurotransmitters* at each synapse effectively gives a different **weight** to the signal coming from each neighbour.
3. **Combines** the weighted signals (a linear combination), and can add some **bias** of its own.
4. **Processes** the result with a non-linear **activation**, and passes the output on to other neurons, for which it becomes an input.

Depending on the task, these weights get **adjusted**, so that the organism can correctly interpret what is being communicated to it.

Say the neuron receives signals x_1, x_2, x_3, x_4 from four neurons, with weights w_1, w_2, w_3, w_4 and own bias w_0. Then

$$h_1 = w_0 + x_1 w_1 + x_2 w_2 + x_3 w_3 + x_4 w_4, \qquad y_1 = a(h_1)$$



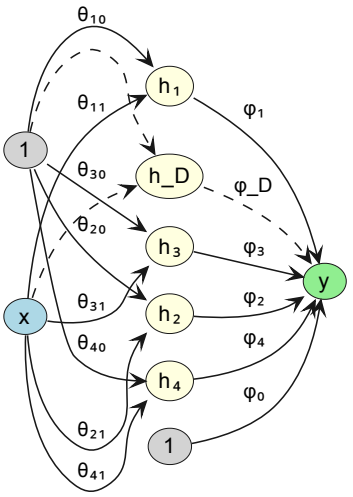
Now go back to the network from Section 5. The hidden unit h_1 receives x and the constant 1, gives them weights, sums them, applies a, and its output becomes an input for the next neuron (here the output unit). **It is exactly the same picture.**

> **Note for Review:** Think of a neuron as a tiny committee member: it listens to all of its inputs, trusts some more than others (the weights), adds its own mood (the bias), and then decides how loudly to speak up (the activation).

Biology	Artificial neuron
Signals from other neurons	Inputs $x_1, \dots, x_4$
Neurotransmitter density at each synapse	Weights $w_1, \dots, w_4$
Neuron's own offset	Bias w_0
Combining the incoming signals	Sum $h_1 = w_0 + \sum_i x_i w_i$
Non-linear response	Activation $a(h_1)$
Signal passed on to the next neurons	Output y_1
Adjusting synapses for the task	Learning (adjusting the weights)

8. The Shallow Neural Network

Now take one input x (plus the bias unit) and one **layer of D neurons**. Each neuron gets the same inputs but uses **its own weights**, so each processes the input differently.



The d-th hidden unit (neuron) computes

$$h_d = a\left(\theta_{d0} + x\,\theta_{d1}\right), \qquad d = 1, \ldots, D$$

and the output is

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \cdots + \phi_D h_D$$

Different activations? In principle each neuron could use its own activation (say a sigmoid in one and a ReLU in another), but in practice **all neurons in a layer use the same activation function**.

Why add more neurons? More neurons means more learnable parameters. With a single hidden neuron there are only four parameters ($\theta_{10}, \theta_{11}, \phi_0, \phi_1$), and four numbers cannot represent a complex boundary. So we add neurons depending on the complexity of the model we need and on how many data points we have.

For one input and D hidden units the parameter count is

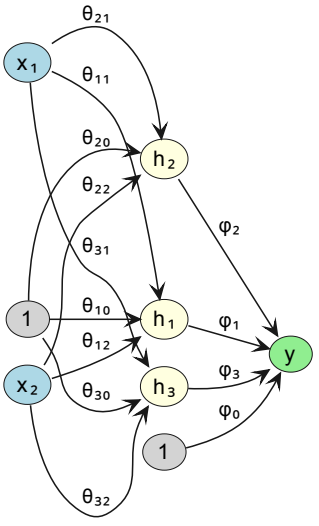
$$\underbrace{2D}_{\theta_{d0},\,\theta_{d1}} + \underbrace{(D+1)}_{\phi_0,\ldots,\phi_D} = 3D + 1$$

(Check: $D = 1$ gives 4 and $D = 3$ gives 10, as before.)

Because there is only one hidden layer, this is called a **shallow neural network**.

9. More Than One Input Feature

With two input features x_1, x_2 the idea is the same: **each neuron receives all the inputs**, and each connection has its own weight. Take three hidden neurons:



Each hidden unit is

$$h_1 = a\left(\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2\right)$$

$$h_2 = a\left(\theta_{20} + \theta_{21}x_1 + \theta_{22}x_2\right)$$

$$h_3 = a\left(\theta_{30} + \theta_{31}x_1 + \theta_{32}x_2\right)$$

and the output multiplies each h_d by ϕ_d and adds the bias ϕ_0:

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

This way we can include **as many input features as we want**, and **as many neurons as we want**.

Parameter count for this network: hidden layer $3 \times 3 = 9$, output layer 4, total 13. In general, for n_{in} inputs and D hidden units:

$$D\left(n_{\text{in}} + 1\right) + (D + 1)$$

10. Layers and Terminology

Take the network of Section 9 and name its parts:

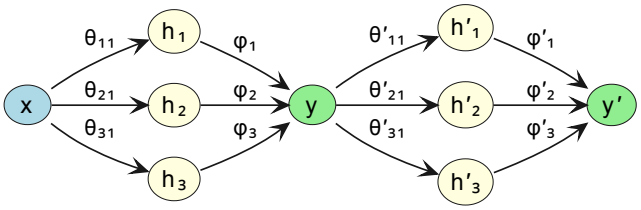
Layer	What it is	In the Section 9 network
Input layer	The input features, plus the bias unit	x_1, x_2 and the constant 1
Hidden layer	Intermediate quantities derived from the inputs, used later to build the output	3 neurons h_1, h_2, h_3
Output layer	The final prediction	1 unit y

They are called **hidden** because they are neither inputs nor outputs: they live in between.

- **Shallow neural network:** a network with a single hidden layer.
- **Deep neural network:** a network with more hidden layers.
- You can add as many hidden layers (and as many neurons per layer) as the data set demands.

11. Going Deep by Stacking

The "naive" way to go deeper is to chain two shallow networks. With one input x and no bias drawn for the time being (to keep the figure tidy), the first network has 3 neurons and produces y. The second network treats y as **its** input, has 3 neurons of its own, and produces y'.



First network:

$$h_1 = a(\theta_{10} + x\theta_{11}), \quad h_2 = a(\theta_{20} + x\theta_{21}), \quad h_3 = a(\theta_{30} + x\theta_{31})$$

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

Second network (its input is y):

$$h_1' = a(\theta_{10}' + y\,\theta_{11}'), \quad h_2' = a(\theta_{20}' + y\,\theta_{21}'), \quad h_3' = a(\theta_{30}' + y\,\theta_{31}')$$

$$y' = \phi_0' + \phi_1' h_1' + \phi_2' h_2' + \phi_3' h_3'$$

This is not the conventional way to build a deep network. We are throwing the rich hidden vector (h_1, h_2, h_3) away and passing only the **single number** y forward.

> **Note for Review:** *Pro-Tip:* Think of the stacked version as sending a whole orchestra's performance through a single microphone before the next stage hears it. Conventional deep networks instead pass the **entire hidden layer** to the next layer, as the next section shows.

12. The Conventional Deep Network

In a conventional deep network, the **outputs of all neurons in one layer become the inputs of every neuron in the next layer**. Because every neuron is connected to all neurons in the previous layer, such a network is called a **fully connected neural network**.

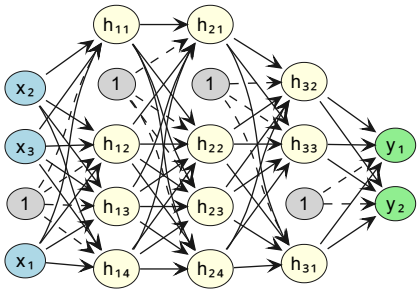
Example used here:

Layer	Contents
Input layer	3 features x_1, x_2, x_3 (+ bias unit)
1st hidden	4 neurons $h_{11}, h_{12}, h_{13}, h_{14}$
2nd hidden	4 neurons $h_{21}, h_{22}, h_{23}, h_{24}$
3rd hidden	3 neurons h_{31}, h_{32}, h_{33}
Output layer	2 outputs y_1, y_2

Notation: h_{kj} is the j-th neuron of the k-th hidden layer. The vector $\mathbf{h}_1$ collects the outputs of layer 1, $\mathbf{h}_2$ those of layer 2, $\mathbf{h}_3$ those of layer 3, and $\mathbf{y}$ is the output.

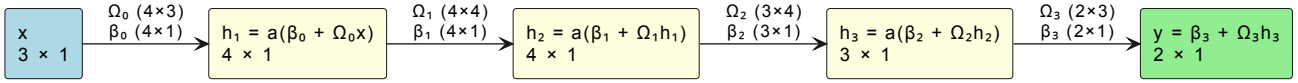
Several outputs are natural: a regression target may be a vector, or there may be several classes (say two) and we want a score for each.

Legend for the figure below: gray = bias unit, blue = input, yellow = hidden neurons, green = outputs. A solid arrow is a weight θ (matrix Ω), a dotted arrow from a gray node is a bias.



> **Note for Review:** Each arrow in the figure is one trainable parameter, so you can verify the parameter count of Section 15 by counting arrows layer by layer (bias arrows included): $16 + 20 + 15 + 8 = 59$.

A compact "layer by layer" view of the same computation, with the shape of each quantity:



13. Matrix Notation

Writing every neuron separately becomes unmanageable, so we vectorize. We derive it for the first hidden layer.

Step 1 — one neuron at a time. Input vector and the weights of neuron 1:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \qquad \boldsymbol{\theta}_1 = \begin{bmatrix} \theta_{11} \\ \theta_{12} \\ \theta_{13} \end{bmatrix}$$

$$h_{11} = a\left(\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2 + \theta_{13}x_3\right) = a\left(\theta_{10} + \boldsymbol{\theta}_1^T \mathbf{x}\right)$$

The sum of products is just the **inner product** of $\boldsymbol{\theta}_1$ and $\mathbf{x}$. The other three neurons are identical, with their own weight vectors:

$$h_{12} = a\left(\theta_{20} + \theta_{21}x_1 + \theta_{22}x_2 + \theta_{23}x_3\right) = a\left(\theta_{20} + \boldsymbol{\theta}_2^T \mathbf{x}\right), \qquad \boldsymbol{\theta}_2 = \begin{bmatrix} \theta_{21} \\ \theta_{22} \\ \theta_{23} \end{bmatrix}$$

$$h_{13} = a\left(\theta_{30} + \boldsymbol{\theta}_3^T \mathbf{x}\right), \qquad h_{14} = a\left(\theta_{40} + \boldsymbol{\theta}_4^T \mathbf{x}\right)$$

Note that $\mathbf{x}$ is the same in all four.

Step 2 — stack the four neurons into a vector.

$$\mathbf{h}_1 = \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{14} \end{bmatrix} = a\left(\begin{bmatrix} \theta_{10} + \boldsymbol{\theta}_1^T \mathbf{x} \\ \theta_{20} + \boldsymbol{\theta}_2^T \mathbf{x} \\ \theta_{30} + \boldsymbol{\theta}_3^T \mathbf{x} \\ \theta_{40} + \boldsymbol{\theta}_4^T \mathbf{x} \end{bmatrix} \right)$$

(The activation a is applied **element-wise**.)

Step 3 — split the biases from the weighted parts.

$$\mathbf{h}_1 = a\left(\begin{bmatrix} \theta_{10} \\ \theta_{20} \\ \theta_{30} \\ \theta_{40} \end{bmatrix} + \begin{bmatrix} \boldsymbol{\theta}_1^T \mathbf{x} \\ \boldsymbol{\theta}_2^T \mathbf{x} \\ \boldsymbol{\theta}_3^T \mathbf{x} \\ \boldsymbol{\theta}_4^T \mathbf{x} \end{bmatrix} \right)$$

The first vector is 4×1 and is called $\boldsymbol{\beta}_0$ (the bias vector of layer 1).

Step 4 — pull the common factor $\mathbf{x}$ out. Since $\mathbf{x}$ appears in all four terms, the second vector is a matrix times $\mathbf{x}$:

$$\mathbf{h}_1 = a\left(\boldsymbol{\beta}_0 + \begin{bmatrix} \boldsymbol{\theta}_1^T \\ \boldsymbol{\theta}_2^T \\ \boldsymbol{\theta}_3^T \\ \boldsymbol{\theta}_4^T \end{bmatrix} \mathbf{x} \right) = a\left(\boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathbf{x}\right) \in \mathrm{R}^{4\times 1}$$

where the weight matrix has one **row per neuron**:

$$\boldsymbol{\Omega}_0 = \begin{bmatrix} \theta_{11} & \theta_{12} & \theta_{13} \\ \theta_{21} & \theta_{22} & \theta_{23} \\ \theta_{31} & \theta_{32} & \theta_{33} \\ \theta_{41} & \theta_{42} & \theta_{43} \end{bmatrix} \in \mathrm{R}^{4\times 3}$$

> **Note for Review:** On the board the fourth row was first forgotten, then added. $\boldsymbol{\Omega}_0$ must be 4×3: 4 neurons in the layer (rows), 3 inputs feeding it (columns).

Step 5 — the other layers. The output of one layer is the input of the next, so we simply replace $\mathbf{x}$ by $\mathbf{h}_1$ (and the shapes change):

- **Second hidden layer** (4 neurons; previous layer has 4 neurons):

$$\mathbf{h}_2 = a\left(\boldsymbol{\beta}_1 + \boldsymbol{\Omega}_1 \mathbf{h}_1\right), \qquad \boldsymbol{\beta}_1 \in \mathrm{R}^{4\times 1}, \quad \boldsymbol{\Omega}_1 \in \mathrm{R}^{4\times 4}$$

- **Third hidden layer** (3 neurons; previous layer has 4):

$$\mathbf{h}_3 = a\left(\boldsymbol{\beta}_2 + \boldsymbol{\Omega}_2 \mathbf{h}_2\right), \qquad \boldsymbol{\beta}_2 \in \mathrm{R}^{3\times 1}, \quad \boldsymbol{\Omega}_2 \in \mathrm{R}^{3\times 4}$$

- **Output layer** (2 units; previous layer has 3). Here we choose **not to apply an activation** in the last layer:

$$\mathbf{y} = \boldsymbol{\beta}_3 + \boldsymbol{\Omega}_3 \mathbf{h}_3, \qquad \boldsymbol{\beta}_3 \in \mathrm{R}^2, \quad \boldsymbol{\Omega}_3 \in \mathrm{R}^{2\times 3}$$

Shape rule to remember:

$$\boldsymbol{\Omega}_k \in \mathrm{R}^{(\text{neurons in layer } k+1) \times (\text{neurons in layer } k)}, \qquad \boldsymbol{\beta}_k \in \mathrm{R}^{(\text{neurons in layer } k+1) \times 1}$$

where "layer 0" is the input layer.

Whole network in one line (this is the function $f[\mathbf{x}, \boldsymbol{\phi}]$):

$$\mathbf{y} = \boldsymbol{\beta}_3 + \boldsymbol{\Omega}_3\, a(\boldsymbol{\beta}_2 + \boldsymbol{\Omega}_2\, a(\boldsymbol{\beta}_1 + \boldsymbol{\Omega}_1\, a(\boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0 \mathbf{x})))$$

14. A Numeric Walk-Through of One Layer

Check the matrix form of layer 1 with real numbers. Take

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}, \qquad \boldsymbol{\Omega}_0 = \begin{bmatrix} 1 & 0 & 2 \\ -1 & 1 & 0 \\ 0 & 2 & 1 \\ 1 & -1 & 1 \end{bmatrix}, \qquad \boldsymbol{\beta}_0 = \begin{bmatrix} 2 \\ 0 \\ -1 \\ 1 \end{bmatrix}$$

Step 1 — matrix-vector product $\boldsymbol{\Omega}_0\mathbf{x}$, row by row (each row is an inner product):

$$\boldsymbol{\Omega}_0\mathbf{x} = \begin{bmatrix} 1\cdot 1 + 0\cdot 2 + 2\cdot(-1) \\ (-1)\cdot 1 + 1\cdot 2 + 0\cdot(-1) \\ 0\cdot 1 + 2\cdot 2 + 1\cdot(-1) \\ 1\cdot 1 + (-1)\cdot 2 + 1\cdot(-1) \end{bmatrix} = \begin{bmatrix} 1+0-2 \\ -1+2+0 \\ 0+4-1 \\ 1-2-1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \\ 3 \\ -2 \end{bmatrix}$$

Step 2 — add the bias vector (pre-activations):

$$\boldsymbol{\beta}_0 + \boldsymbol{\Omega}_0\mathbf{x} = \begin{bmatrix} 2 \\ 0 \\ -1 \\ 1 \end{bmatrix} + \begin{bmatrix} -1 \\ 1 \\ 3 \\ -2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 2 \\ -1 \end{bmatrix}$$

Step 3 — apply ReLU element-wise:

$$\mathbf{h}_1 = a\left(\begin{bmatrix} 1 \\ 1 \\ 2 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} \max(0,1) \\ \max(0,1) \\ \max(0,2) \\ \max(0,-1) \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 2 \\ 0 \end{bmatrix}$$

The fourth neuron produced a negative pre-activation, so ReLU silenced it: it contributes nothing to the next layer for this particular input.

> **Note for Review:** The result $\mathbf{h}_1$ is a 4×1 vector, which is now the input of layer 2. There $\boldsymbol{\Omega}_1$ must be 4×4 so that $\boldsymbol{\Omega}_1\mathbf{h}_1$ is 4×1. If the inner dimensions do not match, something is wrong with your shapes.

15. Counting Trainable Parameters

The full parameter set of the deep network is

$$\boldsymbol{\phi} = \{\boldsymbol{\beta}_0, \boldsymbol{\beta}_1, \boldsymbol{\beta}_2, \boldsymbol{\beta}_3, \boldsymbol{\Omega}_0, \boldsymbol{\Omega}_1, \boldsymbol{\Omega}_2, \boldsymbol{\Omega}_3\}$$

Parameter group	Shape	Count
$\boldsymbol{\beta}_0$	4×1	4
$\boldsymbol{\beta}_1$	4×1	4
$\boldsymbol{\beta}_2$	3×1	3
$\boldsymbol{\beta}_3$	2×1	2
Biases total		**13**
$\boldsymbol{\Omega}_0$	4×3	12
$\boldsymbol{\Omega}_1$	4×4	16
$\boldsymbol{\Omega}_2$	3×4	12
$\boldsymbol{\Omega}_3$	2×3	6
Weights total		**46**

$$13 + 46 = 59 \text{ trainable parameters}$$

This number is the **capacity** of the network.

Same count, neuron by neuron. Each neuron has one weight per incoming neuron plus one bias:

Layer	Neurons	Weights per neuron (incl. bias)	Parameters
1st hidden	4	$3 + 1 = 4$	$4 \times 4 = 16$
2nd hidden	4	$4 + 1 = 5$	$4 \times 5 = 20$
3rd hidden	3	$4 + 1 = 5$	$3 \times 5 = 15$
Output	2	$3 + 1 = 4$	$2 \times 4 = 8$
Total			**59**

Both ways of counting agree: $16 + 20 + 15 + 8 = 59$.

General formula for layer widths n_0 (inputs), $n_1, \ldots, n_L$:

$$\#\text{params} = \sum_{k=1}^{L} (n_k\, n_{k-1} + n_k) = \sum_{k=1}^{L} n_k\, (n_{k-1} + 1)$$

> **Note for Review:** The lecture subtitle mentions "359 trainable weights". That is a transcription slip: the board computation is $13 + 46 = 59$.

With this basic idea of stacking many neurons in a layer and many layers on top of one another, you can build networks with thousands of layers and millions of neurons per layer.

16. Quick Review Sheet

Concept	Formula / Fact
Model	$y = f[x, \boldsymbol{\phi}]$
ReLU	$a(z) = \max(0, z)$, with a kink at $z = 0$
Hidden unit (1 input)	$h_d = a(\theta_{d0} + x\theta_{d1})$
Shallow net output	$y = \phi_0 + \sum_{d=1}^{D} \phi_d h_d$
Kink location	$x = -\theta_{d0}/\theta_{d1}$
Training objective	$\min_{\boldsymbol{\phi}} \frac{1}{n} \sum_i (f[x_i, \boldsymbol{\phi}] - y_i)^2$
One layer, vector form	$\mathbf{h} = a(\boldsymbol{\beta} + \boldsymbol{\Omega}\, \mathbf{h}_{\text{prev}})$
Shape rule	$\boldsymbol{\Omega}_k$: (neurons out) × (neurons in); $\boldsymbol{\beta}_k$: (neurons out) × 1
Fully connected	each neuron sees every output of the previous layer
Shallow vs deep	1 hidden layer vs more than 1
Capacity	number of trainable parameters (59 in the example)

17. Practice Questions

1. Counting bends. A 1-D network has D ReLU hidden units, $y = \phi_0 + \sum_{d=1}^{D} \phi_d \,\mathrm{ReLU}(\theta_{d0} + x\theta_{d1})$. A colleague claims it can produce a function with $D + 5$ linear pieces if you choose the parameters cleverly. Do you agree? Argue using the kink locations. Then explain what happens to the number of pieces if two hidden units happen to have the same kink position, and what happens if you replace ReLU by the identity function.

2. Layer budget. A fully connected network has 10 inputs, hidden layers of widths 20, 15 and 5, and 3 outputs. (a) Write the shape of every $\boldsymbol{\Omega}_k$ and $\boldsymbol{\beta}_k$. (b) Which layer holds the most parameters? (c) If you may add **exactly one** neuron to one hidden layer, which choice adds the most new parameters, and which adds the fewest? Justify with the shape rule.

3. The bottleneck. In Section 11 two shallow networks are chained through the scalar y. Suppose two different inputs $x_a \neq x_b$ happen to give the same value $y(x_a) = y(x_b)$ in the first network. Can the second network ever distinguish them? Compare this with a conventional deep network that passes the whole vector $\mathbf{h}_1 \in \mathrm{R}^3$ forward. What does this tell you about why width matters in the middle of a network?

4. Silent neurons. In Section 14 the fourth neuron of layer 1 outputs 0 for the given $\mathbf{x}$. (a) For that input, what is the effect of the weights $\theta_{41}, \theta_{42}, \theta_{43}$ on the final output $\mathbf{y}$ (use the chain of equations in Section 13)? (b) If a neuron produced 0 for **every** training example, what does it contribute to the model, and what does that say about the effective capacity of the network compared with the raw parameter count?

5. Reverse engineering. A fully connected network has 3 inputs, two hidden layers of **equal** width w, and 1 output, and it has exactly 73 trainable parameters (weights and biases). Find w. Then say how many parameters you would have if you added a third hidden layer of the same width w.